Session Sync Skill — Bidirectional Claude Code Session Import/Export via SSH

Revision: 1 **Last modified:** 2026-07-13T00:00:00Z **Status:** active

Purpose

Syncs ALL Claude Code project data — memories, session transcripts, settings, agent definitions, remember-logs, and handoff documents — between the SAME project on a REMOTE host and the local machine. Supports pull (remote→local), push (local→remote), and bidirectional sync.

After running, the local project can CONTINUE work EXACTLY where the remote session left off — same memories, same session history, same settings. This is the bridge between multi-track workstations (§11.4.187).

When to use

- **Starting work on a new machine** — pull all session state from the primary workstation so you resume exactly where you left off.
- **Switching between workstations** — push your session state back before leaving, pull when you arrive.
- **After a remote session completes major work** — pull the updated memories and handoff docs so the local project knows the latest state.
- **Before/after a rebuild cycle** — sync session state to keep all tracks aligned.

How to invoke

This skill provides the `session-sync.sh` script at: `constitution/skills/session-sync/session-sync.sh`

```
# Pull everything from remote workstation (DEFAULT):
bash constitution/skills/session-sync/session-sync.sh user@host /path/to/project

# Push local state to remote:
bash constitution/skills/session-sync/session-sync.sh --push user@host /path/to/project

# Bidirectional two-way sync:
bash constitution/skills/session-sync/session-sync.sh --bidirectional user@host /path/to/project

# Quick mode (recent sessions only, faster):
bash constitution/skills/session-sync/session-sync.sh --quick user@host /path/to/project

# Preview without copying:
bash constitution/skills/session-sync/session-sync.sh --dry-run user@host /path/to/project
```

What it syncs

Data	Description
Memories	Project-specific memory files (`.md` with frontmatter)
Session JSONL	Full Claude Code session transcripts
Session dirs	Per-session working directories
Settings	`.claude/settings.json` + `settings.local.json`
Agents	`.claude/agents/` — custom agent definitions
Remember logs	`.remember/` — session handoff + daily logs
Handoff docs	`docs/SESSION_RESUME.md` + `docs/CONTINUATION.md`
Provider config	Global `CLAUDE.md` , `history.jsonl` , `settings.json`

Options

Option	Effect
`--pull`	Pull from remote to local (DEFAULT)
`--push`	Push from local to remote
`--bidirectional`	Pull first, then push (full two-way)
`--quick`	Only recent sessions + memories (faster)
`--recent N`	Number of recent sessions in quick mode (default: 3)
`--dry-run`	Preview without copying
`--skip-confirm`	Skip push confirmation prompts
`--no-sessions`	Skip session JSONL transcripts
`--no-session-dirs`	Skip session directories

Cross-references

- constitution/Constitution.md §11.4.187 — multi-track ruler orchestration
- constitution/Constitution.md §11.4.176 — multi-track work-division
- constitution/Constitution.md §11.4.131 — standing session-resumption file
- constitution/Constitution.md §12.10 — CONTINUATION.md maintenance
- constitution/scripts/multitrack/ — multi-track tooling
- constitution/scripts/post_update_hook.sh — auto-registration hook (§11.4.164)

Dependencies

- `ssh` — key-based access to remote host
- `rsync` — efficient file transfer
- bash 4+

How it works

1. Derives Claude Code project slugs from both paths (same algorithm Claude uses: absolute path with `/` replaced by `-`)
2. Validates SSH connectivity + remote project existence
3. Executes 7 sync phases: memories → sessions → session-dirs → settings+agents → remember-logs → handoff-docs → provider-config
4. Prints summary with OK/Failed/Skipped counts
5. Exit 0 = all synced, 1 = partial errors, 2 = blocked

The slug algorithm ensures the correct Claude Code project directory is found regardless of mount point differences between hosts.

Registration

This skill is auto-registered by `constitution/scripts/post_update_hook.sh` (§11.4.164). On every constitution pull, the hook detects new/changed skills and symlinks them into the consuming project's `skills/` directory.